

Decorator Design Pattern

In this activity, I learned how the Decorator Design Pattern adds responsibilities to an object without modifying its original implementation. The HTTP client application required logging, caching, retry, and token-counting features. Using inheritance for every possible combination could create many classes, resulting in duplicated code and making the system harder to maintain. The activity showed how four features could result in sixteen possible combinations.

To solve this problem, I used the `HttpClient` interface, the `BaseHttpClient` class, and the `HttpDecorator` class to create a flexible wrapping structure. I completed the `RetryDecorator` so that it attempts a request up to three times and throws the exception after the final failed attempt. I also completed the `TokenCounterDecorator` by counting the words in the response. Finally, I tested different decorator arrangements and examined how the order of logging and caching affected the request process.

Through this activity, I understood how decorators can add features dynamically through composition instead of inheritance. Each decorator has a specific responsibility, allowing logging, caching, retry, and token counting to be combined based on the system's requirements. I also learned that the order of decorators matters because it affects how the request and response pass through the pipeline. This pattern makes the HTTP client easier to extend because additional behavior can be introduced without creating a separate class for every feature combination.